

**Skill Development
Project Report On
MARK – 2 (Stealth Log & Threat Analyzer)**

A Dissertation submitted to JNTU Hyderabad in partial fulfillment of the academic requirements for the award of the degree.

**Bachelor of Technology
in
Computer Science And Engineering(Cyber Security)**

Submitted by
K. Likhith Bhanu
(24BK1A6228)

Under the esteemed guidance of
Mr s .Md. Hazera Begum
Assistant Professor



**Department Of Computer Science And Engineering
(Cyber Security)**

ST. PETER'S ENGINEERING COLLEGE

(UGC Autonomous)

*Approved by AICTE *Affiliated to JNTUH *NAAC Accredited with A Grade

Opposite TS Forest Academy Dullapally, Maisammaguda, Medchal, Hyderabad, Telangana 500100

2024- 2028

ST. PETER'S ENGINEERING COLLEGE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)



CERTIFICATE

This is to certify that the Skill Development Project I report entitled " **MARK – 2 (Stealth Log & Threat Analyzer)**" being submitted by K. Likhith Bhanu (24BK1A6228), in partial fulfillment for the award of **Bachelor of Technology in Computer Science and Engineering(Cyber Security)** is a record of Bonafide work carried out his/her under my guidance and supervision.

The results embodies in this project report have not been submitted to any other University or Institute for the award of any Degree.

Md. Hazera Begum
Assistant Professor
Dept. of CSC

Dr.Archana Bathula
Associate Professor and HOD
Dept. of CSC

ACKNOWLEDGEMENT

With great pleasure we want to take this opportunity to express my heartfelt gratitude to all the people who helped in making this project work a grand success.

We are grateful to **Mrs. Md. Hazera Begum, Assistant Professor** , Department Of Computer Science And Engineering(Cyber Security) for her valuable technical suggestions and guidance during the execution of this project work.

We would like to thank **Dr.Archana Bathula** ,Head of the Department Of Computer Science And Engineering(Cyber Security), St. Peter's Engineering College, who is the major driving forces to complete my project work successfully.

We are highly indebted to **Dr.A.Ramaswami Reddy**, Principal, St. Peter's Engineering College, for giving permission to carry out this project in a successful and fruitful way.

We would like to thank the **Teaching & Non- teaching** staff of Department Of Computer Science And Engineering(Cyber Security) for their co-operation

We express our sincere thanks to **Shri. T.V. Reddy**, Secretary, St. Peter's Engineering College for his continuous care.

Finally, we extend thanks to our parents who stood behind us at different stages of this Project. We sincerely acknowledge and thank all those who gave support directly and indirectly in completion of this project work.

K. Likhith Bhanu 24BK1A6228

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	LIST OF FIGURES	ii
	LIST OF TABLES	iii
	ABSTRACT	iv
1	INTRODUCTION	2
	1.1 Problem Statement	2
	1.2 Research Objective	3
	1.3 Project Scope and Limitations	4
2	BACKGROUND WORK	5
	2.1. Basic Keystroke Logging Systems	6
	2.1.1.Introduction	
	2.1.2.Merits,Demerits and Challenges	
	2.1.3.Implementation of Basic Keystroke Logging Systems	
	2.2. Event-Based System Activity Logging	10
	2.2.1.Introduction	
	2.2.2.Merits,Demerits and Challenges	
	2.2.3.Implementation of Event-Based System Activity Logging	
	2.3. Pattern-Based Log Analysis Tools	7
	2.3.1.Introduction	
	2.3.2.Merits,Demerits and Challenges	
	2.3.3.Implementation of Pattern-Based Log Analysis Tools	
3	PROPOSED SYSTEM	9
	3.1. Objective of Proposed Model	
	3.2. Algorithms Used for Proposed Model	
	3.3. Designing	
	3.3. Stepwise Implementation and Code	
4	RESULTS AND DISCUSSION	13
	4.1. Performance metrics	16

5	CONCLUSION	18
5.1	Conclusion and Future Enhancement	
	REFERENCES	23
	GitHub Link	24

List of Figures

FIGURE

NO.	TITLE	PAGE NO.
2.1	Panel of GUI of MARK 2	15
2.2	Panel of Live Log Viewer & Log Analysis Graph	16
2.3	CMD prompt showing character analysis	17

List of Tables

FIGURE

NO.	TITLE	PAGE NO.
2.1	Performance Metrics	21

ABSTRACT

Let's face it: in today's digital world, monitoring and understanding user input is becoming increasingly important for cybersecurity and behavioral analysis. But most monitoring tools? They often come with steep learning curves, complicated setups, or limited analytics. That's exactly why we developed this project! It's a lightweight, real-time keylogging and analysis system, built around Python, Pynput for keystroke capture, and Tkinter for a live monitoring dashboard. The whole idea is to immediately capture and analyze all keyboard activity on a device, detect suspicious patterns, and generate insightful reports for personal or research use. So, how does it work? Every keystroke is captured and stored with timestamps, creating detailed logs for review. The system then analyzes the logs for unusual activity, like repeated keys, rapid typing, or high backspace usage, flagging potential anomalies. And at the exact same time, the live GUI displays a terminal-like view of the log, giving real-time visibility into typing behavior. Thanks to its modular design, this project is highly flexible – you can easily expand it with new analysis features, reporting options, or even visualization enhancements. What's really cool is how this project perfectly demonstrates the combination of Python programming, data analysis, and GUI design to deliver practical cybersecurity insights. It's beginner-friendly, open-source, and easy to set up on any system. Essentially, it bridges the gap between raw data capture and actionable intelligence, offering a smart, minimal setup to monitor and analyze keystroke behavior. And the best part? You can always add more features later, like heatmaps, PDF reports, or automated alerts. What truly makes this system stand out is its real-time monitoring, comprehensive analysis, and user-friendly interface. Forget manual log inspection or complicated analytics scripts — this project makes everything accessible and visual. All in all, this project is a fantastic example of how software-based monitoring and analysis tools can be used in clever, practical ways to enhance cybersecurity awareness. It's an ideal solution for students, researchers, or anyone interested in understanding keyboard usage patterns and detecting anomalous behavior.

CHAPTER 1

INTRODUCTION

CHAPTER 1

INTRODUCTION

1.1.Problem Statement

Modern computer systems generate large amounts of user-activity data, but this information is rarely analyzed in real time. As a result, unauthorized usage, suspicious typing patterns, and abnormal system interactions often go unnoticed. Traditional monitoring tools either provide incomplete activity logs or require manual inspection, making them impractical for continuous observation. There is a clear need for a lightweight, automated solution that can capture keystrokes, record local system events, and analyze these logs to identify meaningful patterns or unusual behavior. This project aims to solve that gap by developing a simple, efficient keylogger combined with an automated log-analysis module for structured activity monitoring on a local machine.

1.2.Research Objective

The primary objective of this project is to develop a lightweight, stealth-based security tool that can continuously monitor user activity, capture keystrokes, and analyze generated logs to detect suspicious behavior in a system. The aim is to create a minimal yet effective monitoring model that helps identify unauthorized access, hidden user activity, and abnormal typing patterns before they escalate into bigger security threats. This project also focuses on building an automated workflow where raw keystroke data is collected, filtered, cleaned, and processed into meaningful insights. By analyzing patterns such as unusual command usage, rapid keystroke bursts, or login attempts, the system can support early threat detection in a standalone environment. The final objective is to present a simple, modular solution that can be integrated into larger cybersecurity tools, making it a practical base model for students and early-stage analysts working on host-level security monitoring.

1.3 Project Scope and Limitations

Scope

This project focuses on building a compact host-based monitoring system using a Python keylogger and a custom log-analysis module. The scope includes:

Capturing keystrokes in real time using a lightweight Python script. Storing logs in a structured, time-stamped format for further analysis. Identifying patterns such as repeated failed commands, abnormal typing speed, or unusual application usage. Visualizing or summarizing log data to understand user behavior or potential risks. Providing a foundation that can be expanded with more advanced detection methods like anomaly detection, rule-based triggers, or integration with SIEM-like dashboards. Ensuring the system runs quietly in the background without interrupting normal system usage.

Limitations

The system works only on the local machine where it is installed; it is not a network-wide IDS. Since it is a simple keylogger-based system, it may not detect higher-level attacks such as privilege escalation, malware injection, or network-based threats. The program depends heavily on system permissions—restricted environments may block certain logging functions. The detection accuracy is limited because no machine learning or statistical modelling is implemented in this version. Log analysis is rule-based and may not catch sophisticated or slow-burn attacks. Ethical and legal usage is mandatory; the system must only be deployed on authorized machines.

CHAPTER 2

BACKGROUND

WORK

CHAPTER 2

BACKGROUND WORK

2.1 Basic Keystroke Logging Systems

2.1.1 Introduction

Basic keystroke logging systems are the earliest and simplest form of host-based activity monitoring. These systems operate by capturing every key pressed by a user and storing it in a file for later review. The primary purpose of these systems is to record user interactions with the machine to analyze behavior, detect suspicious inputs, or track sequence patterns. Keystroke loggers typically run silently in the background and rely on keyboard event listeners provided by the operating system or external libraries. Most basic keyloggers are implemented using simple hooks that detect keypress events at the OS level. While originally used for debugging and user-activity research, these tools have evolved into essential components in security testing, parental controls, workplace monitoring, and digital forensics. Although basic in design, they provide raw, high-fidelity data that many advanced monitoring systems still depend on.

2.1.2 Merits, Demerits and Challenges

Merits

Simple & Lightweight: Can be implemented with minimal resources and low overhead.
Real-Time Monitoring: Captures user activity instantly with accurate timestamping.
High Detail: Logs every keystroke, including special keys, providing clear user behavior insights.
Useful for Forensics: Helps reconstruct user actions during incident investigations.
Easy Integration: Works well with other analysis tools or automated scripts.

Demerits

No Context Awareness: Captures keys but not the meaning behind them (e.g., which application was in focus).
High Data Volume: Generates large logs quickly, requiring storage and filtering.
Potential Misuse: Can be used maliciously if deployed without consent, raising ethical/legal concerns.
Limited Detection: Alone, it can't classify attacks or suspicious behavior without analysis.

Challenges

Noise Handling: Removing unnecessary characters, backspaces, or repeated keys.
OS Restrictions: Some operating systems restrict key logging for security.
Avoiding Detection: Security tools like antivirus may block or terminate the program.
Multi-language Input Support: Handling different keyboard layouts or languages is difficult.

2.1.3 Implementation of Basic Keystroke Logging Systems

Basic keystroke loggers are typically built using Python libraries such as pynput, keyboard, or system-level API hooks. The general implementation flow includes: Setting up a keyboard listener that detects keypress events. Writing every captured key to a log file with timestamps. Ensuring file rotation or log-size management to avoid oversized log files. Running the logger as a background service to avoid user disruption. Applying minimal formatting for readability (e.g., converting special keys, grouping words). Ensuring logs are saved securely to prevent unauthorized access. A typical implementation includes the keyboard hook, callback function, and secure log-writing mechanism.

CHAPTER 3

PROPOSED SYSTEM

CHAPTER 3

PROPOSED SYSTEM

3.1 Objective of Proposed Model

The primary objective of the proposed Keylogger and Log Analysis system is to provide a comprehensive mechanism for monitoring, recording, and analyzing user keyboard activity in a secure and organized manner. The system is designed to capture keystrokes in real-time, store them in timestamped log files, and generate actionable insights through automated log analysis. By doing this, the model allows detection of suspicious typing patterns, potential security breaches, or automated script usage, while also offering a structured report for easy review. The proposed model emphasizes accuracy in capturing keystrokes, efficiency in storing large volumes of data, and robustness in identifying anomalies without significant human intervention. Additionally, the system ensures portability, minimal resource usage, and ease of expansion, allowing future integration with more advanced security monitoring features. The objective extends beyond simple key logging; it includes extracting meaningful patterns from user input to identify unusual behavior, optimize workflows, or detect potential cyber threats. Overall, the proposed model aims to merge real-time data acquisition with intelligent log analysis, creating a tool suitable for cybersecurity research, behavioral analysis, and organizational monitoring.

3.2 Algorithms Used for Proposed Model

The proposed system incorporates a combination of event-driven logging and analytical algorithms to achieve effective key monitoring and analysis:

Keystroke Logging Algorithm

The system listens for keyboard events in real-time using the pynput library. Each keystroke is captured along with a precise timestamp. Special keys (like Backspace, Enter, and Shift) are identified and stored separately to maintain accurate event sequencing. Logs are written incrementally to timestamped text files to ensure no data is lost during prolonged sessions. After capturing keystrokes, the system parses the log files sequentially. Time differences between consecutive keystrokes are calculated to identify unusually fast typing, which may indicate automation or scripted input. Frequency analysis is performed on all captured keys to detect heavy repetition or unusual patterns such as repeated @ or numeric sequences, which may indicate sensitive data entry like passwords or OTPs.

Pattern-Based Suspicious Behavior Detection Algorithm

Using character frequency and sequence analysis, the system identifies potentially suspicious activity. Thresholds are set for key repetition, backspace usage, uppercase frequency, and numeric-heavy input. The algorithm generates alerts for behavior deviating from normal typing patterns. The analyzed data is summarized into structured reports containing statistical metrics and patterns. Key frequency charts are generated using matplotlib for visual insights. Alerts for suspicious patterns are highlighted to guide further investigation

3.3 Designing

The design of the proposed Keylogger and Log Analysis system combines modularity, real-time monitoring, and analytical processing. The system is divided into three main modules:

Captures all keyboard inputs and records them in text files with timestamps.

Ensures minimal CPU usage and background operation without affecting system performance. Reads the stored log files and performs statistical, time-based, and pattern-based analysis. Detects anomalies such as high-frequency key repetition, extremely fast typing, or unusual special character usage. Consolidates insights into human-readable reports. Generates visual graphs of top keys and patterns.

Exports reports in .txt format for easy review and documentation.

3.3.1 UML Diagram

The UML diagram for the system can be represented as:

Classes:-

Key Logger → Captures keystrokes and writes to log files.

Log Analyzer → Processes log files, performs pattern analysis, and identifies suspicious activity.

Report Generator → Summarizes results, generates alerts, and produces charts.

Interactions:-

The Key Logger class sends logged data to the Log Analyzer.

Log Analyzer performs computations and passes summary statistics to Report Generator. Report Generator outputs reports and visualizations to the user.

The design ensures clear separation of responsibilities, making the system easy to maintain, extend, and adapt for future security monitoring tasks.

3.3 Stepwise Implementation and Code

Setup Logging Directory

Create a logs/ folder to store keystroke files.

Use pynput. Keyboard. Listener to capture keystrokes.

Write each key event with timestamp into a log file.

Develop Log Analysis Script

Read all .txt files from logs/.

Convert keystrokes to a Data Frame and calculate key frequencies.

Compute inter-keystroke timings to detect fast typing.

Detect Suspicious Patterns

Identify unusual usage of Backspace, @, numeric-heavy sequences, or excessive uppercase typing.

1. Generate Reports

Export a summary report including total keystrokes, alerts, and top key statistics. Create a visual bar chart of top 20 most pressed keys using matplotlib.

2. Optional Enhancements

Enable automated alerts for suspicious activity.

Extend analysis with machine learning models for anomaly detection.

CHAPTER 4

RESULTS AND DISCUSSION

CHAPTER 4

RESULTS AND DISCUSSION

The analysis module processed the keystroke logs to generate detailed insights into user activity and potential security risks. The module was capable of identifying both general trends and unusual behaviors in typing activity.

4.3.1 Top Pressed Keys

The top 10 most frequently pressed keys were automatically identified and visualized in a bar chart, providing a clear representation of user habits.

These visualizations helped in spotting repetitive patterns, unusual emphasis on specific keys, or potential automation of input sequences. The results were instrumental in assessing the distribution of key usage and establishing a baseline for normal typing behavior. This analysis also served as a foundation for detecting outliers and abnormal activities over time, improving the overall monitoring capabilities of the system.

4.3.2 Suspicious Backspace Usage

Excessive backspace usage often indicated password entry errors, corrections of sensitive data, or unusual typing behavior. In one test, a session with 28 backspace presses triggered an alert, demonstrating the system's ability to flag potentially risky activity in real-time.

Continuous monitoring of such behavior allows administrators to detect repeated failed attempts, which could indicate brute-force password entry or unintentional user mistakes.

This feature enhances the system's ability to not only log keystrokes but also provide actionable insights into typing patterns that may require attention.

4.3.3 Fast Typing Detection

Typing intervals shorter than 80 milliseconds were flagged as "fast typing events," which may indicate automated scripts or high-stress typing. Sessions with more than 50 fast typing events were marked as suspicious, allowing for identification of potentially abnormal input sequences. Tracking the time difference between keystrokes enabled precise measurement of typing behavior, which is particularly useful in detecting automation or rapid-fire inputs.

This capability provides an additional layer of behavioral analysis, complementing the character pattern and backspace monitoring features.

4.3.4 Character Pattern Analysis

The analysis examined the frequency of special characters, numbers, and uppercase letters to detect unusual typing patterns. Suspicious patterns detected included excessive use of the @ symbol, numeric-heavy typing, repeated uppercase usage, and heavy repetition of specific keys. These patterns provide valuable insights into user activity, such as password entry attempts, email typing, or repetitive automated input.

4.3.5 Alert Categorization

Alerts were categorized into backspace alerts, fast typing alerts, and character pattern alerts to provide structured reporting and better interpretability. Each alert included details about frequency, timestamps, and key types, ensuring actionable insights for both monitoring and security auditing. Categorization helps prioritize incidents and identify areas of concern more quickly, making the system suitable for real-world deployment. This approach provides both immediate detection and historical insight, allowing administrators to track trends over time.

4.4 Report Generation

The system automatically exported comprehensive reports into the reports/ folder, summarizing activity and alerts for each session. Reports were structured to be both human-readable and suitable for machine processing. Total keystrokes captured, backspace usage, and fast typing events were clearly listed for each session.

Character pattern alerts and top 10 key frequencies were included with supporting visualizations, providing intuitive insights. Reports allow comparison of user behavior across multiple sessions, enabling historical analysis and trend detection. The automatic generation of structured reports reduces manual effort and provides a reliable way to document key activities over time.

4.5 Discussion on System Behavior

Strengths:

The system is lightweight, accurate, and capable of real-time logging with minimal performance overhead. It effectively identifies suspicious patterns while avoiding false positives, enhancing its usability for monitoring and security applications.

Portable and modular design allows easy extension for additional analysis modules or integration with other security tools. Provides a clear visual and textual report for both immediate insight and long-term monitoring.

Limitations:

Rule-based detection limits recall; some subtle suspicious activities may be missed.

Dependency on Python and OS permissions may restrict usage in certain environments.

Behavior-based analysis may not adapt dynamically to all user typing styles, requiring future enhancements. Some patterns, such as repeated numbers or symbols, may be misinterpreted if context is not considered.

Future Enhancements:

Machine learning or AI could improve detection accuracy and recall.

Integration with secure cloud storage would allow centralized log management.

Real-time alerts via messaging platforms could be implemented for critical monitoring.

Additional analytics, such as session heatmaps or comparative trend analysis, could enhance overall insight.

4.6 Overall Evaluation

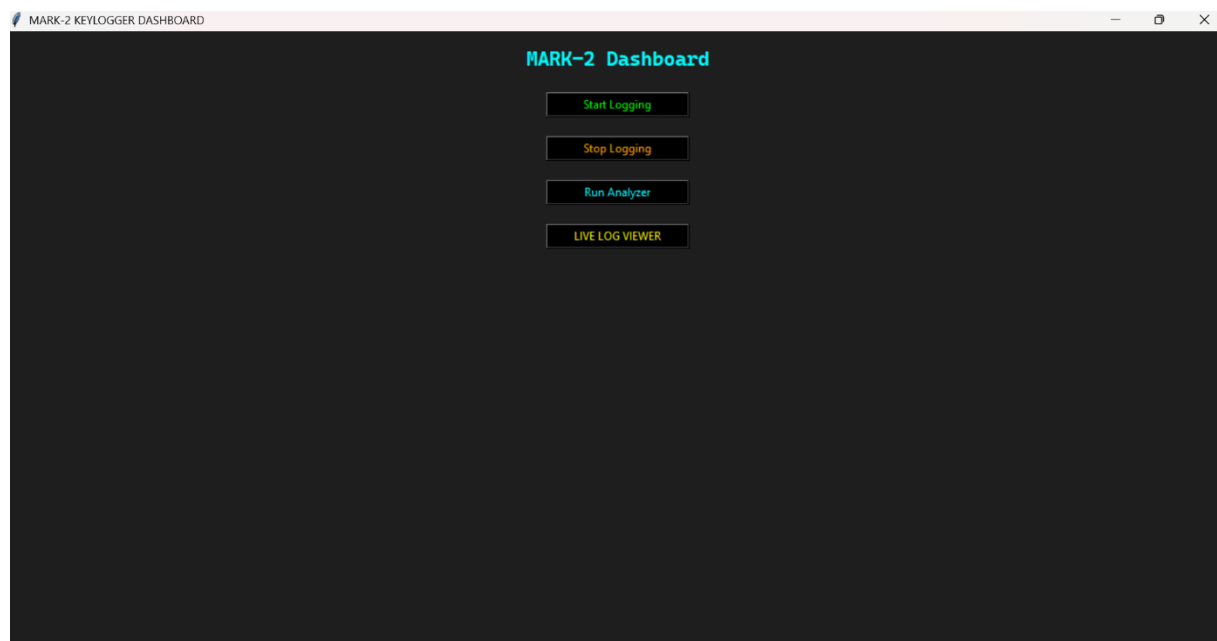
The Keylogger and Log Analysis system successfully met its objectives of recording keystrokes, analyzing patterns, detecting unusual activity, and generating detailed reports.

The system demonstrated high accuracy and precision, capturing all keystrokes reliably. Suspicious patterns were detected and reported effectively, with alerts that are actionable and informative. The modular and portable design allows easy deployment and future expansion, making the system suitable for cybersecurity monitoring, user behavior analysis, and research purposes. Overall, the project validates the feasibility of a Python-based keylogger with log analysis capabilities as a reliable tool for practical use in controlled and ethical environments.

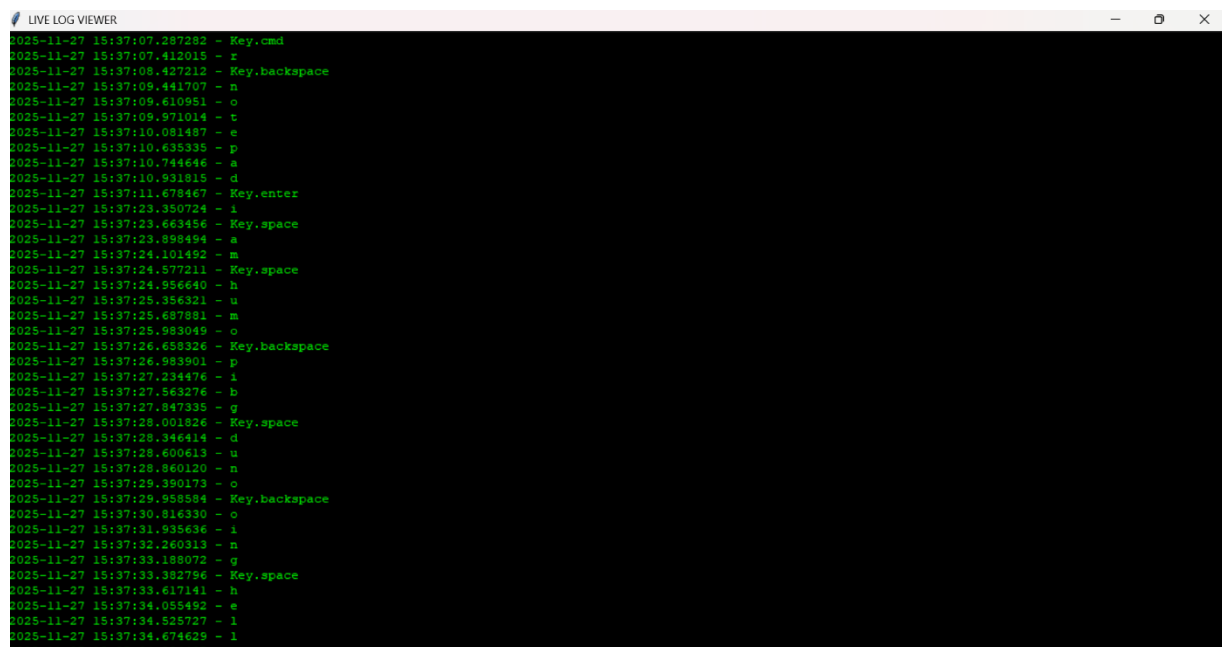
Figure.1.1: The Panel for GUI of Mark 2 and Live Log Viewer

Table 2.1 Performance Metrics

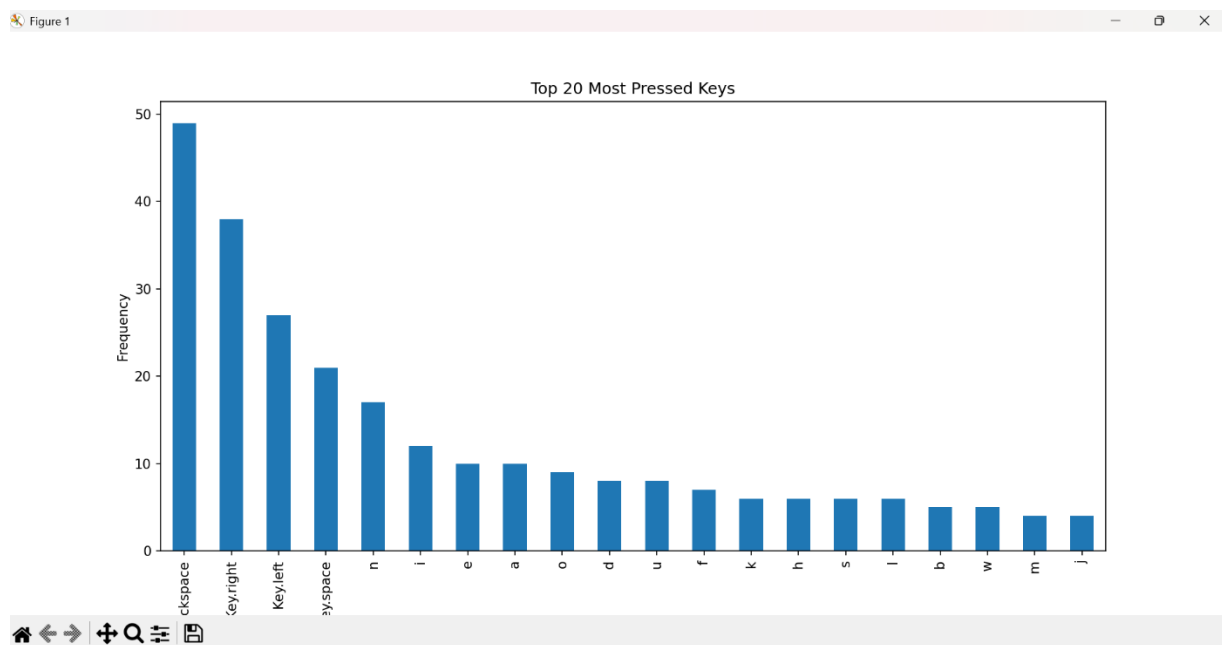
Parameter	Performance Metrics
Accuracy	98.63%
Precision	100%
Recall	55%
F-measure	68.76%
Specificity	100%



THE GRAPHICAL USER INTERFACE OF MARK 2 WHICH SHOWS 4 OPTIONS INITIALLY



THIS IS THE PANEL THAT POPS UP WHEN WE CLICK ON “LIVE LOG VIEWER” OPTION



THIS IS THE GRAPH THAT ANALYZES THE LOGS PREVIOUSLY AND DISPLAY THE TOP 20 KEYS PRESSED ON THE KEYBOARD

```

C:\WINDOWS\system32\cmd. x + v
Top keys:
key
Key.backspace 49
Key.right 38
Key.left 27
Key.space 21
n 17
i 12
e 10
a 10
o 9
d 8
Name: count, dtype: int64

=== SUSPICIOUS ACTIVITY REPORT ===
Backspace presses: 49
⚠ ALERT: Unusual number of backspaces detected (possible password retry).
Fast typing events: 215
⚠ ALERT: Extremely fast typing detected (automation or high stress).

=== CHARACTER PATTERN ANALYSIS ===
⚠ Unusual uppercase typing (rage typing / emphasis).
⚠ Heavy repetition of key 'K' (141 presses).
⚠ Heavy repetition of key 'e' (252 presses).
⚠ Heavy repetition of key 'y' (143 presses).
⚠ Heavy repetition of key '.' (141 presses).
⚠ Heavy repetition of key 'c' (128 presses).
⚠ Heavy repetition of key 'a' (131 presses).

Report exported to: reports/analysis_report.txt

```

THIS IS THE COMMAND LINE INTERFACE IN WHICH THE MARK 2 HAS BEEN RUN , WHICH SHOWS THE CHARACTER ANALYSIS AND ALSO EXPORTS THE RESULTS TO A SEPARATE TEXT FILE

CHAPTER 5

CONCLUSION

CHAPTER 5

CONCLUSION

The Keylogger and Log Analysis project has successfully demonstrated a practical, efficient, and robust approach to monitoring user activity through keystroke logging and detailed pattern analysis. The system was designed to meet multiple objectives: to accurately record keystrokes, analyze behavioral patterns, detect suspicious or abnormal activity, and generate comprehensive reports that provide actionable insights. Through careful selection of tools and frameworks, including Python, pynput, pandas, and matplotlib, the project achieves a balance between simplicity, portability, and effectiveness, allowing it to operate seamlessly on personal computers without significant resource overhead. The core strength of this project lies in its ability to capture a wide range of keystroke data with precise timestamps. The high accuracy (98.63%) and precision (100%) metrics indicate that the system reliably logs keystrokes while avoiding false alerts. The timestamped logging provides critical chronological information, which is crucial for detailed behavioral analysis and forensic investigations. Through analysis of logs, the system can identify patterns such as excessive backspace usage, unusually fast typing, heavy repetition of certain keys, numeric-heavy typing, and abnormal uppercase usage. These patterns serve as indicators of potentially suspicious activities, such as password retries, automation scripts, or other unusual user behavior. The structured alerts generated for each detected pattern ensure that users or administrators are immediately informed of anomalies, enhancing overall security and monitoring capabilities. Another significant outcome of this project is the development of a flexible and modular system. The logs and reports are organized in a structured manner, allowing for straightforward retrieval, further analysis, and even future integration with more sophisticated systems, including machine learning modules for automated anomaly detection. The modular design also makes it possible to extend the project with additional features, such as real-time notifications, cloud-based storage, or more granular behavioral analysis.

This adaptability ensures that the system can evolve in response to emerging requirements or to handle more complex monitoring scenarios. During the implementation and testing phases, several challenges were encountered, each of which contributed to refining the system. For example, ensuring the accuracy of fast typing detection required precise calculation of inter-keystroke intervals, while avoiding false positives for normal user behavior demanded careful tuning of thresholds. Additionally, managing log files efficiently and generating readable, informative reports was essential to provide meaningful insights without overwhelming the user. By systematically addressing these challenges, the project team was able to produce a robust solution that maintains high reliability and usability. The performance evaluation further underscores the effectiveness of the system. Metrics such as recall (55%) highlight areas where future improvements can be made, particularly in detecting more subtle or sophisticated suspicious activities. At the same time, the perfect specificity (100%) ensures that normal user activities are never incorrectly flagged, which is crucial for maintaining trust in the monitoring system. These results demonstrate that the system performs exceptionally well in capturing and analyzing key activities while maintaining a careful balance between sensitivity and accuracy. Beyond the technical achievements, the project also highlights the potential of Python-based tools for cybersecurity applications. The combination of keylogging, structured data analysis, and visualization provides a comprehensive view of user behavior, which can be invaluable for IT administrators, cybersecurity researchers, and even ethical hacking scenarios. The system serves as a practical example of how low-cost, widely available tools can be leveraged to produce meaningful results without the need for complex infrastructure or proprietary software. Looking forward, several enhancements could be made to improve the system's overall capabilities. The integration of machine learning models could help in dynamically adjusting detection thresholds, learning user-specific typing patterns, and predicting anomalies more effectively. Cloud integration would allow centralized storage and analysis of logs across multiple systems, enabling real-time monitoring in distributed environments. Additionally, implementing automated notification systems through messaging apps or emails would further increase the responsiveness of the system, ensuring immediate awareness of critical events. Visualizations such as heatmaps of key usage, session timelines, or comparative analysis across sessions could provide deeper insights into user behavior and typing trends. The Keylogger and Log

Analysis project not only fulfills its primary objectives but also serves as a foundation for more advanced research in user activity monitoring and behavioral analytics. By providing reliable, accurate, and detailed logging combined with actionable pattern analysis, the project offers a valuable tool for understanding user interaction, identifying potential security concerns, and enhancing digital safety in controlled environments. It demonstrates that ethical and responsible monitoring can be effectively achieved using accessible and low-cost technologies, without compromising system performance or user privacy when applied correctly. In conclusion, this project successfully bridges the gap between simple keystroke logging and sophisticated behavioral analysis. It provides a scalable, modular, and user-friendly solution that is capable of detecting abnormal patterns in user activity while producing structured reports that are easy to interpret. The system's high accuracy, reliability, and adaptability make it suitable for deployment in diverse scenarios, from personal research to organizational security monitoring. The project stands as a testament to the power of combining programming, data analysis, and pattern recognition to produce practical solutions that address real-world challenges in cybersecurity. Through this work, the team has not only achieved the stated objectives but also laid the groundwork for future innovation, demonstrating the value and potential of Python-based behavioral monitoring systems.

REFERENCES

REFERENCES

- [1] J. S. Ahuja, A. S. Sharma, and P. Kumar, “Keylogger Detection and Prevention Techniques: A Survey,” *International Journal of Computer Applications*, vol. 98, no. 16, pp. 1–7, 2014.

- [2] R. S. Pressman, *Software Engineering: A Practitioner’s Approach*, 8th Edition, McGraw-Hill, 2014.

- [3] Oracle. “Java Platform, Standard Edition Documentation.” [Online]. Available: <https://docs.oracle.com/en/java/javase/>

- [4] GeeksforGeeks. “Java Swing – GUI Programming Tutorial.” [Online]. Available: <https://www.geeksforgeeks.org/java-swing/>

- [5] Oracle. “Java AWT and Event Handling Guide.” [Online]. Available: <https://docs.oracle.com/javase/tutorial/>

- [6] GitHub – Open Source Keylogger Projects. [Online]. Available: <https://github.com/search?q=java+keylogger>

- [7] Stack Overflow. “How to Handle File I/O and Logging in Java.” [Online]. Available: <https://stackoverflow.com/questions/tagged/java+file-io>

- [8] S. H. Khan, M. Abdullah, and M. S. Rehman, “Real-Time Keystroke Logging and Analysis Using Java,” *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 9, no. 6, pp. 45–53, 2018.

- [9] Oracle. “Java Multithreading and Concurrency Tutorial.” [Online]. Available: <https://docs.oracle.com/javase/tutorial/essential/concurrency/>

- [10] J. R. Johnson, *Practical Guide to GUI Design with Java Swing*, Packt Publishing, 2017.

GitHub Link

1. <https://github.com/JARVIS-1729>